

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 3 : 딥러닝

DAY3. 딥러닝 데이터 처리와 데이터셋

목 차

1. 데이터셋 이해		3
2. MNIST 데이터셋 딥러닝 실습		15
3. CIFAR-10 데이터셋 딥러닝 실습		29
4. 다차원 배열의 계산		35

1. 데이터셋 이해

1) 데이터의 중요성

(1) 데이터의 중요성

- 딥러닝은 기계학습의 분야 중 하나
 - 특정 분야의 전문가
- 입력과 결과를 전달해 컴퓨터가 패턴을 찾고 특정 프로그램을 만드는 과정
 - 최대한 많은 데이터를 경험해야만 전문가가 됨
- 2016년 구글 딥마인드 알파고 이후 국내외에서 인공지능 열풍
 - 실험 단계를 넘어 상용화 단계로 일상생활 곳곳에서 활용되고 있음
- 인공지능 적용으로 비생산적인 반복 작업 자동화 등 **업무 효율성 증가**
- 비즈니스 의사결정 및 예측 등 **상당한 효과 기대**

딥러닝은 인공지능을 만드는 기술 중 하나이다. 사람도 어떤 분야의 전문가가 되기 위해서는 많은 경험과 공부가 필요하듯이, 딥러닝 모델도 많은 데이터를 학습해야 좋은 성능을 낼 수 있다.

컴퓨터는 입력 데이터와 정답 데이터를 함께 제공받으면 그 안에서 규칙과 패턴을 찾아 학습한다. 학습한 내용을 바탕으로 새로운 데이터에 대한 예측이나 분류를 수행하게 된다.

2016년 알파고 이후 인공지능 기술은 급속히 발전하였으며, 현재는 음성인식, 얼굴인식, 자율주행, 의료 진단 등 다양한 분야에서 사용되고 있다. 인공지능은 반복적인 업무를 자동화하고 미래를 예측하는 데 도움을 주기 때문에 많은 기업과 기관에서 활용하고 있다.

따라서 인공지능의 성능을 높이기 위해서는 좋은 품질의 데이터를 많이 확보하는 것이 매우 중요하다.

1) 데이터의 중요성

(2) 인공지능을 활용하려면?

- 수많은 데이터를 수집하여 컴퓨터에게 **입력, 정답 데이터**를 알려주고, 학습시켜야 함



(3) 인공지능 활용 예시

- 피서철 드론을 활용한 물에 빠진 사람 실시간 탐지
 - 인공지능 **학습**을 하려면?
 - 물에 빠져 위험한 사람의 이미지 **데이터 필요**
 - 학습의 **정확도**를 올리려면?
 - 노인, 어린이, 물에 빠졌지만 위험하지 않은 사람 등 **수많은 데이터 필요**

인공지능 모델을 만들기 위해서는 먼저 데이터를 수집해야 한다. 데이터가 없으면 학습 자체가 불가능하다.

수집된 데이터는 분석 과정을 거쳐 어떤 특징이 있는지 확인하고, 학습에 적합하도록 가공한다. 이러한 과정을 통해 딥러닝이 학습할 수 있는 데이터셋이 만들어진다.

예를 들어 드론으로 물에 빠진 사람을 탐지하는 인공지능을 만든다면, 물에 빠진 사람의 사진뿐 아니라 어린이, 성인, 노인, 수영하는 사람 등 다양한 상황의 사진이 필요하다.

다양한 데이터를 학습할수록 인공지능은 실제 상황에서도 더 정확하게 판단할 수 있게 된다.

2) 데이터셋의 종류

(1) 데이터셋의 종류

- 학습을 위한 훈련 데이터셋
 - 문제를 풀고 정답을 알려줘 무엇이 틀린지 학습
- 검증을 위한 검증 데이터셋
 - 학습 중 학습이 잘 되는지 평가
- 테스트를 위한 시험(테스트) 데이터셋
 - 학습 완료 후 학습이 잘 되었는지 평가

(2) 올해 수능을 가장 잘 보기 위한 데이터셋 분류하기

- 현재 보유중인 데이터는 총 6개
 - 모의고사 1회~5회
 - 작년 수능 기출문제 1회
- 모두 훈련 데이터로 사용
 - 테스트 없이 공부만 시켰기에 올해 수능을 잘 볼 수 있을지 장담하지 못함



딥러닝에서는 데이터를 목적에 따라 세 가지로 나누어 사용한다.

훈련 데이터셋은 모델이 공부하는 데 사용하는 데이터이다. 정답을 함께 제공하여 모델이 규칙을 학습하도록 한다.

검증 데이터셋은 학습 도중 모델이 잘 학습되고 있는지 확인하는 데 사용한다.

시험 데이터셋은 학습이 모두 끝난 후 최종 성능을 평가하는 데 사용한다.

학생이 공부할 때 문제집으로 공부하고, 모의고사로 실력을 점검한 후, 실제 수능을 보는 과정과 비슷하다고 생각하면 이해하기 쉽다.

2) 데이터셋의 종류

(2) 올해 수능을 가장 잘 보기 위한 데이터셋 분류하기

- 모의고사로만 훈련, 작년 수능문제는 공부를 잘 했는지 테스트를 위한 시험셋으로 사용
 - 작년 수능문제를 통해 올해 수능을 어느 정도로 볼지 가늠할 수 있음



- 모의고사를 훈련/검증셋으로 다시 분류, 작년 수능문제는 시험셋으로 사용
 - 검증셋을 활용해 공부를 잘 하고 있는지 중간에 확인하며 학습 가능



모든 데이터를 학습에만 사용하면 실제 성능을 정확하게 평가하기 어렵다.

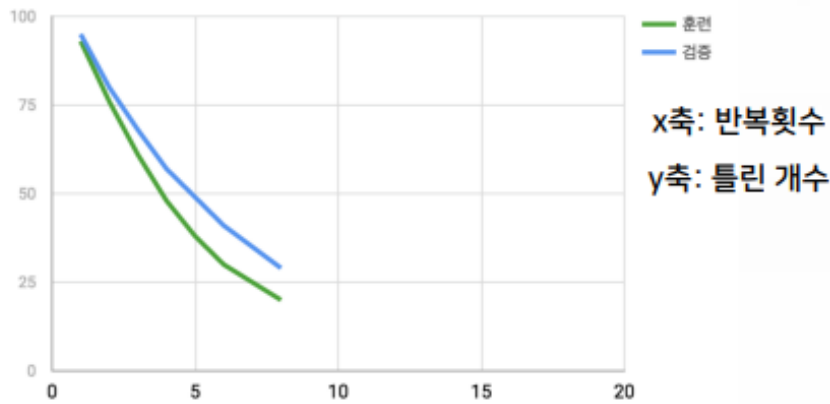
예를 들어 모의고사 문제만 공부하고 작년 수능 문제는 시험용으로 남겨두면 자신의 실력을 객관적으로 확인할 수 있다.

또한 학습 중간에도 실력을 확인하기 위해 일부 데이터를 검증용으로 따로 분리할 수 있다.

이처럼 훈련셋, 검증셋, 시험셋으로 나누면 모델이 실제로 얼마나 잘 동작하는지 정확하게 평가할 수 있다.

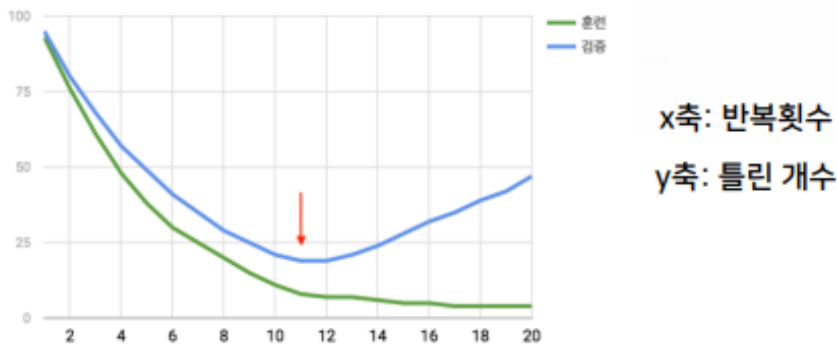
2) 데이터셋의 종류

(1) 훈련 데이터셋 vs 검증 데이터셋



■ 언더피팅(underfitting)

- 더 반복해서 학습시키면 성능이 더 좋아질 수 있음



■ 오버피팅(overfitting)

- 검증셋은 어느 순간부터 안 좋아지기 시작함
- 검증 데이터셋을 따로 추가해 학습이 잘되고 있는지 평가

언더피팅은 모델이 충분히 학습되지 않은 상태를 말한다.

이 경우 훈련 데이터와 검증 데이터 모두에서 성능이 좋지 않다. 학습 횟수를 늘리거나 모델을 개선하면 성능이 좋아질 수 있다.

오버피팅은 학습 데이터를 너무 많이 외워버린 상태를 말한다.

훈련 데이터에서는 매우 좋은 결과가 나오지만 새로운 데이터에서는 성능이 떨어진다.

그래서 검증 데이터셋을 사용하여 학습이 잘 진행되고 있는지 지속적으로 확인해야 한다.

1) 딥러닝의 학습과정

(1) 딥러닝의 학습과정



딥러닝 모델을 만드는 과정은 크게 다섯 단계로 이루어진다.

먼저 데이터를 준비하고, 학습에 사용할 모델을 설계한다.

그 다음 학습 방법을 설정한 후 실제로 학습을 진행한다.

학습이 끝나면 모델의 성능을 평가하고 분석한다.

마지막으로 완성된 모델을 실제 서비스나 프로그램에 활용한다.

이 과정이 딥러닝 프로젝트의 기본적인 흐름이다.

2) 딥러닝의 모델 준비

(1) 딥러닝의 모델 준비

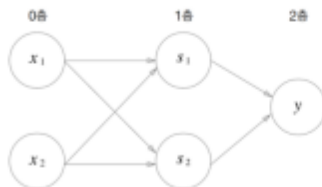
- 딥러닝 모델의 종류는 여러 가지가 있음
- MLP(Multi Layer Perceptron, 다층 퍼셉트론) 활용
 - 순차적으로 층을 쌓아나가기 때문에 Sequential 모델 사용
- 하나의 층을 쌓아 연결을 해주는 Dense(densely connected Neural Network)
 - Dense(출력 뉴런 수, input_dim=입력 뉴런 수, activation=활성화 함수)

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation
model = Sequential()
model.add(Dense(20, input_dim=2, activation='sigmoid'))
```

출력

입력

▪ XOR 예시



0층 : 입력이 2개,
1층 : 0층 출력 2개가 입력
2층 : 출력 1개

```
model = Sequential()
model.add(Dense(2, input_dim=2, activation='sigmoid'))
model.add(Dense(1, activation='sigmoid'))
```

딥러닝에는 여러 종류의 모델이 있지만 이번 강의에서는 MLP(다층 퍼셉트론)를 사용한다.

MLP는 여러 층을 순서대로 연결한 가장 기본적인 인공신경망 구조이다.

Sequential 모델을 이용하여 층을 차례대로 추가할 수 있다.

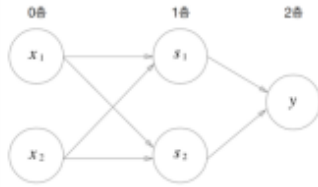
Dense 층은 모든 뉴런이 서로 연결된 완전 연결 신경망을 의미한다.

XOR 문제는 단층 퍼셉트론으로 해결할 수 없기 때문에 여러 층을 사용하는 다층 퍼셉트론의 필요성을 설명할 때 자주 사용된다.

2) 딥러닝의 모델 준비

(1) 딥러닝의 모델 준비

▪ XOR 예시



0층 : 입력이 2개,
1층 : 0층 출력 2개가 입력
2층 : 출력 1개

```
model = Sequential()  
model.add(Dense(2, input_dim=2, activation='sigmoid'))  
model.add(Dense(1, activation='sigmoid'))
```

input_dim을 사용하지 않는 이유?
→ 순차적으로 하나씩 층이 추가가 되기 때문에
자동으로 텐서플로우가 인식 해서 적용함

3) 딥러닝의 학습 과정 준비

(1) 딥러닝의 학습 과정 준비

▪ 완성한 모델을 조립하여 학습하는 방법을 지정

(정답을 찾아가는 방법)

- 학습 할 모델의 손실 함수 및 옵티마이저, 평가지표 등을 설정

모델 구조를 만든 후에는 학습 방법을 지정해야 한다.

이 과정에서 손실 함수(Loss Function), 옵티마이저(Optimizer), 평가 지표(Metric)를 설정한다.

손실 함수는 모델이 얼마나 틀렸는지를 계산한다.

옵티마이저는 틀린 부분을 수정하여 더 좋은 결과를 얻도록 가중치를 조정한다.

평가 지표는 정확도와 같은 성능 수치를 계산하는 역할을 한다.

이러한 설정을 완료해야 모델이 실제로 학습을 시작할 수 있다.

4) 딥러닝의 학습

(1) 준비된 모델 학습

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation

model = Sequential()
model.add(Dense(2, input_dim=2, activation='sigmoid'))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='sgd', metrics=['accuracy'])

hist = model.fit(X_train, Y_train, epochs=200, batch_size=10, validation_data=(X_val, Y_val))
```

모델 학습을 위한 함수

반복횟수

한번에 풀 갯수

검증 데이터셋

(2) 배치사이즈(Batch_size)

- 한 번에 몇 문제씩 풀고 정답을 맞춰볼까?
 - 학습: 문제를 다 풀고 틀린 문제를 맞춰보며 가중치 갱신을 통해 정답을 더 잘 맞출 수 있도록 모델을 향상시켜 나가는 것

문제지

문제1	문제2	문제3	문제4	문제5	문제6	...	문제100
-----	-----	-----	-----	-----	-----	-----	-------

해답지

정답1	정답2	정답3	정답4	정답5	정답6	...	정답100
-----	-----	-----	-----	-----	-----	-----	-------

- Batch_size 10
 - 10문제를 풀고 또 비교하고 가중치 갱신
- Batch_size 100
 - 100문제를 다 푼 다음에 한번만 정답을 비교해서 가중치 갱신

모델 학습은 fit() 함수를 사용하여 수행한다.

Epoch는 전체 데이터를 몇 번 반복해서 학습할지를 의미한다.

Batch Size는 한 번에 몇 개의 데이터를 사용하여 학습할지를 의미한다.

예를 들어 Batch Size가 10이면 10개의 데이터를 처리한 후 가중치를 수정한다.

Batch Size가 100이면 100개의 데이터를 모두 처리한 후 한 번만 가중치를 수정한다.

따라서 Batch Size는 학습 속도와 성능에 큰 영향을 주는 중요한 설정값이다.

4) 딥러닝의 학습

(2) 배치사이즈(Batch_size)

큰 것	VS	작은 것
• 학습 속도 빠름 → 가중치 갱신을 자주 하지 않기 때문 • 많은 메모리, 컴퓨팅 성능 필요 → 한 번에 많은 문제를 풀고 문제에 대한 정답들을 기억 및 학습해야함		• 학습 속도 느림 → 가중치 갱신을 자주 하기 때문 • 많은 메모리, 컴퓨팅 성능 불필요

배치 크기가 크면 한 번에 많은 데이터를 처리하기 때문에 학습 속도가 빠르다.

하지만 많은 데이터를 동시에 저장해야 하므로 메모리가 많이 필요하다.

배치 크기가 작으면 메모리 사용량이 적고 컴퓨터 사양이 낮아도 학습할 수 있다.

반면 가중치를 자주 수정해야 하므로 학습 시간이 길어질 수 있다.

실제 딥러닝에서는 보통 32, 64, 128 정도의 배치 크기를 많이 사용한다.

5) 딥러닝의 평가

(1) 딥러닝의 평가

- 학습 완료된 모델을 시험셋에 적용하여 모델 성능 평가

Loss(손실)

낮을 수록 모델의
성능이 좋음

Accuracy(정확도)

높을 수록 모델의
성능이 좋음

```
loss_and_metrics = model.evaluate(X_test, Y_test, batch_size=1)
print('')
print('loss : ' + str(loss_and_metrics[0]))
print('accuracy : ' + str(loss_and_metrics[1]))
```

모델 학습이 끝나면 시험 데이터셋을 사용하여 성능을 평가한다.

대표적인 평가 지표는 Loss와 Accuracy이다.

Loss는 모델의 오차를 의미하며 낮을수록 좋다.

Accuracy는 정답을 맞춘 비율을 의미하며 높을수록 좋다.

따라서 좋은 모델은 Loss 값은 작고 Accuracy 값은 높게 나타난다.

5) 딥러닝의 평가

(2) 딥러닝의 분석 및 활용

- 분석 및 활용 방법의 대표적 방법은 **그래프로 시각화**하여 활용하는 것
 - 단순한 숫자로 알기 힘든 정보들을 한눈에 파악 할 수 있음

```
import matplotlib.pyplot as plt
fig, loss_ax = plt.subplots()
acc_ax = loss_ax.twinx()

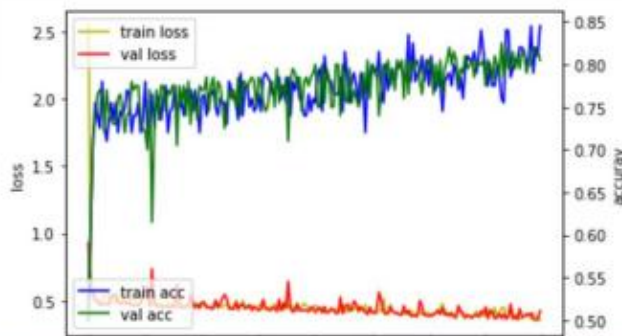
loss_ax.plot(hist.history['loss'], 'y', label='train loss')
loss_ax.plot(hist.history['val_loss'], 'x', label='val loss')

acc_ax.plot(hist.history['accuracy'], 'b', label='train acc')
acc_ax.plot(hist.history['val_accuracy'], 'g', label='val acc')

loss_ax.set_xlabel('epoch')
loss_ax.set_ylabel('loss')
acc_ax.set_ylabel('accuracy')

loss_ax.legend(loc='upper left')
acc_ax.legend(loc='lower left')

plt.show()
```



Accuracy(정확도) : 우상향일수록 좋음

Loss(손실) : 우하향일수록 좋음

학습 결과는 그래프로 시각화하면 이해하기 쉽다.

Loss 그래프는 학습이 잘 진행되면 점점 아래로 내려가는 형태를 보인다.

Accuracy 그래프는 학습이 잘 진행되면 점점 위로 올라가는 형태를 보인다.

그래프를 보면 과적합이나 학습 부족과 같은 문제를 쉽게 발견할 수 있다.

따라서 실제 딥러닝 프로젝트에서는 학습이 끝난 후 반드시 Loss와 Accuracy 그래프를 확인하여 모델 상태를 분석한다.

2. MNIST 데이터셋 딥러닝 실습

2) MNIST 데이터셋 이해

(1) MNIST 데이터셋

- 0부터 9까지의 손글씨 이미지 제공
 - 딥러닝 학습 시 가장 기초 실습 예제 데이터로 활용
- 훈련 데이터 6만개, 테스트 데이터 1만개로 총 7만개의 손글씨 데이터 제공

MNIST 는 딥러닝과 머신러닝을 처음 학습할 때 가장 많이 사용하는 대표적인 데이터셋이다. MNIST 는 미국 국립표준기술연구소(NIST)의 손글씨 숫자 데이터를 가공하여 만든 데이터셋으로, 0 부터 9 까지의 숫자를 사람이 직접 손으로 쓴 이미지를 모아 놓은 것이다.

이 데이터셋은 딥러닝 모델이 숫자를 인식하는 방법을 학습하는 데 사용된다. 예를 들어 사람이 손으로 쓴 숫자 "5" 이미지를 입력하면 컴퓨터가 이것을 숫자 5 라고 판단할 수 있도록 학습시키는 것이다.

MNIST 데이터셋은 총 70,000 개의 손글씨 이미지로 구성되어 있다.

- 훈련 데이터(Training Data): 60,000 개
- 테스트 데이터(Test Data): 10,000 개

훈련 데이터는 모델이 공부하는 데 사용되고, 테스트 데이터는 학습이 끝난 후 성능을 평가하는 데 사용된다.

2) MNIST 데이터셋 이해

(2) MNIST 데이터셋의 장점

- 데이터의 크기가 비교적 크지 않음
- 모델의 학습 결과를 눈으로 쉽게 확인하기 쉬움
- 가로(28) X 세로(28) 크기
- 기본적인 모델로도 성능이 95% 이상 나올 만큼 쉽고 편리한 예제
- 글씨가 써져 있는 부분은 값이 있고, 나머지는 0으로 구성



(3) matplotlib 모듈 사용



MNIST 데이터셋은 딥러닝 입문자가 실습하기에 매우 적합한 데이터셋이다.

첫 번째 이유는 데이터 크기가 적당하기 때문이다.

최근의 이미지 데이터셋은 수백만 장 이상의 이미지가 포함되어 있는 경우가 많다. 그러나 MNIST는 7만 개 정도의 데이터만 가지고 있기 때문에 일반 PC나 노트북에서도 충분히 학습할 수 있다.

두 번째 이유는 결과를 눈으로 쉽게 확인할 수 있기 때문이다.

예를 들어 아래 그림과 같은 숫자가 있다면 사람은 바로 숫자 7이라고 인식할 수 있다.

#####

#

#

#

#

컴퓨터 역시 이러한 숫자 패턴을 학습하게 된다.

MNIST 의 각 이미지는

가로 28 픽셀

세로 28 픽셀

크기로 구성되어 있다.

즉,

$$28 \times 28 = 784$$

개의 픽셀 값을 가진다.

이미지 내부의 각 픽셀은 밝기 값을 저장한다.

예를 들어

0 = 검정색

255 = 흰색

사이의 숫자로 표현된다.

숫자가 적혀 있는 부분은 밝기 값이 존재하고, 배경 부분은 거의 0 에 가까운 값을 가진다.

그래서 실제 데이터를 출력하면 다음과 비슷하게 보인다.

0 0 0 0 0

0 0 0 0 0

0 0 255 0 0

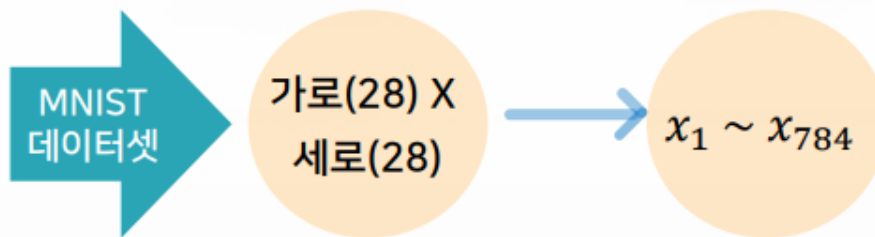
0 255 255 255 0

즉, 컴퓨터는 숫자를 "그림"으로 보는 것이 아니라 수많은 숫자들의 배열로 인식한다.

또한 MNIST 는 비교적 단순한 데이터셋이기 때문에 기본적인 신경망만 사용해도 95% 이상의 높은 정확도를 얻을 수 있다.

1) MNIST 데이터셋을 활용한 딥러닝 실습

(3) 입력 데이터 변환



(4) 정답 데이터 라벨링 변환

- 기존 정답 데이터
 - 0, 1, 2 ... 9 처럼 하나의 숫자
- 딥러닝 모델의 예측 결과
 - 0부터 9 각각 숫자마다 확률을 돌려주기 때문에 **정답 개수만큼 배열형태로 변환**
 - loss 손실값을 줄이는 것을 학습목표로 하기 때문에 여러 개 중에 한 개로 나올 수 있도록 **이상적인 정답 값으로 변환**해주어야 함

```
Y_train = utils.to_categorical(Y_train)
Y_val = utils.to_categorical(Y_val)
Y_test = utils.to_categorical(Y_test)
```

```
print(Y_train[0])
```

```
[0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
```

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

MNIST 이미지 한 장의 크기는

28 × 28

이다.

그러나 기본적인 MLP 신경망은 2 차원 이미지를 그대로 입력받을 수 없다.

그래서 다음과 같이 변환한다.

28 × 28

↓

784

즉,

$x_1, x_2, x_3, \dots, x_{784}$

형태의 1 차원 벡터로 바꾸는 것이다.

예를 들면

```
[  
  [1,2,3],  
  [4,5,6],  
  [7,8,9]  
]
```

를

$[1,2,3,4,5,6,7,8,9]$

로 펼치는 것과 같다.

이를 Flatten 또는 Reshape 라고 부른다.

다음은 정답 데이터 변환이다.

원래 정답은

5

처럼 숫자 하나로 저장되어 있다.

하지만 딥러닝 모델은

0 일 확률

1 일 확률

2 일 확률

...

9 일 확률

총 10 개의 확률을 출력한다.

예를 들어 숫자 5 라면

$[0,0,0,0,0,1,0,0,0,0]$

로 변환한다.

이를 One-Hot Encoding 또는 범주형 인코딩이라고 한다.

코드는 다음과 같다.

```
Y_train = utils.to_categorical(Y_train)
```

숫자 5 는

```
[0,0,0,0,0,1,0,0,0,0]
```

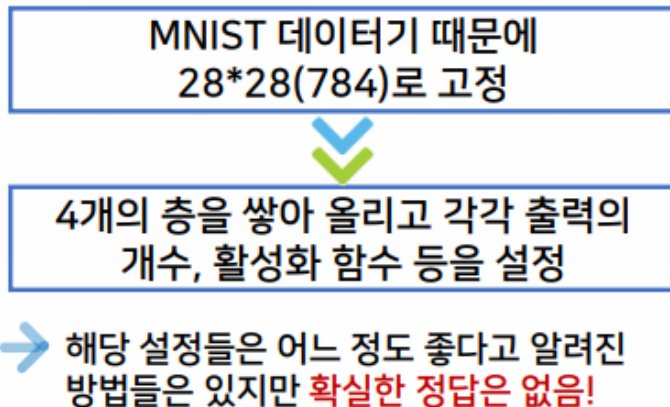
숫자 3 은

```
[0,0,0,1,0,0,0,0,0,0]
```

가 된다.

1) MNIST 데이터셋을 활용한 딥러닝 실습

(5) 딥러닝 학습을 위한 모델 구성



MNIST 이미지는

$$28 \times 28 = 784$$

개의 입력값을 가진다.

따라서 입력층은 784 개의 입력 노드를 가진다.

구조를 그림으로 표현하면 다음과 같다.

입력층

784 개

↓

은닉층 1

512 개

↓

은닉층 2

256 개

↓

은닉층 3

128 개

↓

출력층

10 개

출력층의 10 개 노드는

0
1
2
3
4
5
6
7
8
9

각 숫자일 확률을 의미한다.

예를 들어 출력 결과가

[0.01, 0.02, 0.03, 0.01,
0.01, 0.85, 0.02, 0.03,
0.01, 0.01]

이라면

숫자 5의 확률이 가장 높으므로

예측 결과 = 5

가 된다.

여기서 사용한 **ReLU** 활성화 함수는 학습 속도가 빠르고 성능이 좋아 현재 가장 널리 사용되는 활성화 함수 중 하나이다.

마지막 층의 **Softmax** 함수는 10 개의 출력값을 모두 0~1 사이의 확률로 변환하고, 전체 합이 항상 1이 되도록 만들어 준다.

따라서 MNIST와 같은 **10 개 클래스 분류 문제(Multi-Class Classification)**에서는 Softmax가 가장 많이 사용된다.

1) MNIST 데이터셋을 활용한 딥러닝 실습

(5) 딥러닝 학습을 위한 모델 구성

- Softmax
 - 여러 개 중에 하나를 분류를 해야 되는 문제에서 전체 출력의 데이터들 0과 1사이의 값으로 조정해주는 함수
- Softmax 함수를 사용한 이유
 - 이상적인 정답 데이터를 위해 사용
(softmax는 입력받은 값을 출력으로 0~1사이의 값으로 모두 정규화하며 출력 값들의 총합은 항상 1이 되는 특성을 가진 함수)
 - 여러가지 정답이 있는 카테고리컬 분류 문제이기 때문
 - 만약 둘 중 하나를 나타내는 바이너리 문제면 sigmoid 사용

(1) Softmax 함수란?

딥러닝 모델이 숫자 0~9 를 분류한다고 가정해 보자.

모델의 마지막 층에서는 다음과 같이 10 개의 출력값이 생성된다.

[-2.3, 1.5, 0.8, 3.2, 0.1,
5.6, 2.1, 1.2, -1.0, 0.5]

하지만 이 값들은 확률이 아니라 단순 계산 결과이다.

사람이 보기에는 어떤 숫자인지 알기 어려움

따라서 이 값들을

0~1 사이의 확률값

으로 변환해야 한다.

이때 사용하는 함수가 바로 **Softmax 함수**이다.

(2) Softmax 적용 결과

Softmax 를 적용하면 다음과 같이 변환된다.

[0.001,
0.005,
0.003,
0.020,
0.002,
0.950,
0.010,
0.005,
0.001,
0.003]

이 값들은 모두

0 이상 1 이하

의 값을 가지게 된다.

또한 중요한 특징이 있다.

모든 확률의 합이 항상 1 이 된다.

$$0.001 + 0.005 + \dots \\ = 1.0$$

즉,

각 숫자가 될 확률을 의미하게 된다.

(3) Softmax 가 숫자를 예측하는 방법

예를 들어 결과가 다음과 같다고 가정하자.

[0.01,
0.02,
0.01,
0.03,
0.01,
0.88,
0.01,
0.01,
0.01,
0.01]

이를 표로 나타내면

숫자 확률

0	1%
1	2%
2	1%
3	3%
4	1%
5	88%
6	1%
7	1%
8	1%
9	1%

가 된다.

가장 큰 값은 0.88 이다.

따라서 모델은

"이 이미지는 숫자 5 일 가능성이 가장 높다"

라고 판단한다.

(4) 정답 데이터를 변환하는 이유

MNIST 의 원래 정답 데이터는 다음과 같다.

5

하지만 Softmax 는 10 개의 확률을 출력한다.

따라서 정답도 같은 형태로 만들어야 한다.

예를 들어 숫자 5 는

[0,0,0,0,0,1,0,0,0,0]

으로 변환된다.

이를 One-Hot Encoding

또는

범주형(Categorical) 데이터 변환 이라고 한다.

(5) 왜 Softmax 를 사용하는가?

MNIST 는

0
1
2
3
4
5
6
7
8
9

중 하나를 선택하는 문제이다.

즉, 정답 후보가 여러 개 존재한다.

이를 다중 분류(Multi-Class Classification) 문제라고 부른다.

다중 분류에서는 각 클래스의 확률을 계산해야 하므로 Softmax 가 가장 적합하다.

(6) Softmax 수식

Softmax 의 계산식은 다음과 같다.

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

여기서

- z_i : 출력층에서 계산된 값
- e : 자연상수(약 2.718)
- $P(y_i)$: 해당 클래스의 확률

을 의미한다.

이 수식을 이용하면 모든 출력값이

0 ~ 1 사이

로 변환되고

전체 합은 항상 1 이 된다.

(7) Softmax 대신 Sigmoid 를 사용하는 경우

마지막 부분에서는 둘 중 하나를 선택하는 문제라면 Sigmoid 사용이라고 설명하고 있다.

예를 들어 **MNIST** 0~9 중 하나 선택

→ Softmax 사용

스팸메일 분류

스팸

정상메일

둘 중 하나 선택

→ Sigmoid 사용

고양이/강아지 분류

고양이

강아지

→ Sigmoid 사용

3. CIFAR-10 데이터셋 딥러닝 실습

1) CIFAR-10 데이터셋을 활용한 딥러닝 실습

(1) CIFAR-10 데이터셋

- 인공지능 모델 학습용으로 제공되는 데이터셋
- MNIST 데이터셋보다 난이도가 높음
- 10종류의 컬러 이미지 제공(CIFAR-10)
 - 100종류의 이미지 데이터를 제공하는 CIFAR-10 데이터셋도 있음
- 텐서플로우 모듈에서 손쉽게 다운로드 및 활용 가능

```
cifar10_data = cifar10.load_data()  
((X_train, Y_train), (X_test, Y_test)) = cifar10_data
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170500096/170498071 [=====] - 40s 0us/step

- 이미지의 크기는 32X32X3이며, 10가지 종류의 이미지 제공



훈련 데이터 50,000개

테스트 데이터 10,000개



총 60,000개의 이미지
데이터를 제공

(1) CIFAR-10 데이터셋이란?

CIFAR-10 은 캐나다 토론토 대학교에서 제작한 이미지 데이터셋으로, 총 10 개의 클래스(Class)를 포함하고 있다.

여기서 클래스란 이미지의 종류를 의미한다.

CIFAR-10 이 제공하는 10 개의 클래스는 다음과 같다.

클래스 번호 이미지 종류

0	비행기(Airplane)
1	자동차(Automobile)
2	새(Bird)
3	고양이(Cat)
4	사슴(Deer)
5	개(Dog)
6	개구리(Frog)
7	말(Horse)
8	배(Ship)
9	트럭(Truck)

즉, 모델은 입력된 이미지가 위의 10 가지 종류 중 무엇인지를 맞추도록 학습한다.

예를 들어 자동차 사진이 입력되면 모델은

비행기 : 2%

자동차 : 95%

새 : 1%

...

와 같은 확률을 계산하여 가장 높은 확률인 자동차를 선택한다.

(2) MNIST 보다 어려운 이유

MNIST 는 흑백 손글씨 숫자 이미지였다.

예를 들어 숫자 5 와 숫자 8 을 구분하는 문제였다.

5

8

반면 CIFAR-10 은 실제 사물 사진을 분류해야 한다.

예를 들면

고양이

개

말

사슴

과 같이 서로 비슷한 동물들을 구분해야 한다.

또한 이미지의

- 촬영 위치
- 크기
- 각도
- 밝기
- 배경

이 모두 다르다.

예를 들어 같은 자동차라도

정면 사진

측면 사진

야간 사진

비 오는 날 사진

등이 존재한다.

따라서 CIFAR-10 은 MNIST 보다 훨씬 복잡한 특징을 학습해야 하므로 난이도가 높다.

(3) CIFAR-10 과 CIFAR-100

강의자료에서는 CIFAR-10 뿐만 아니라 CIFAR-100 도 언급하고 있다.

CIFAR-10 은

10 개 클래스

를 제공한다.

반면 CIFAR-100 은 100 개 클래스를 제공한다.

예를 들어 CIFAR-10 은 단순히

개

로 분류하지만

CIFAR-100 은

푸들

시베리안 허스키

골든 리트리버

와 같이 훨씬 세부적으로 분류한다.

따라서 CIFAR-100 이 CIFAR-10 보다 훨씬 어려운 데이터셋이다.

(4) 이미지 크기

MNIST 의 이미지 크기는

28×28

이었다.

반면 CIFAR-10 은

$32 \times 32 \times 3$

크기를 가진다.

여기서 마지막 숫자인 3 은 RGB 색상 채널을 의미한다.

R = Red (빨강)

G = Green (초록)

B = Blue (파랑)

즉,

가로 : 32 픽셀

세로 : 32 픽셀

색상 : RGB 3 채널

로 구성되어 있다.

그래서 실제 입력 데이터의 크기는

$$32 \times 32 \times 3$$

$$= 3072 \text{ 개 값}$$

이 된다.

MNIST 는

$$28 \times 28 = 784 \text{ 개}$$

였으므로 CIFAR-10 이 훨씬 많은 정보를 포함하고 있음을 알 수 있다.

(5) 데이터 개수

CIFAR-10 데이터셋은 총 60,000 장의 이미지를 제공한다.

구분	개수
훈련 데이터	50,000 개
테스트 데이터	10,000 개
총 데이터	60,000 개

훈련 데이터는 모델 학습에 사용된다.

테스트 데이터는 학습이 끝난 후 모델의 성능을 평가하는 데 사용된다.

(7) 그림의 의미

페이지 하단에 표시된 작은 이미지들은 CIFAR-10 에 포함된 실제 샘플 이미지들이다.

이미지를 보면

- 자동차
- 비행기
- 새
- 개
- 고양이
- 말
- 배

등이 포함되어 있다.

딥러닝 모델은 이러한 수만 장의 이미지를 반복적으로 학습하면서 각 클래스의 특징을 스스로 찾게 된다.

예를 들어

고양이

↓

귀 모양

눈 모양

얼굴 형태

를 학습하고,

비행기

↓

날개

동체

꼬리 날개

를 학습하여 새로운 이미지가 들어왔을 때 올바르게 분류하게 된다.

4. 다차원 배열의 계산

1) 행렬의 정의와 연산

(1) 행렬(Matrix)

- 직사각형 형태의 수들의 배열

➡ 이 때, 그 수를 성분(Entry) 또는 원소(Element)라고 함

예

$$\begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}, [8], \begin{bmatrix} -3 & 4 \\ 2 & 1 \\ 0 & 9 \end{bmatrix}$$

(2) 행행렬(Row matrix), 행벡터(Row vector)

- $1 \times m$ 행렬

예

$$[1 \quad 3 \quad -9], [0 \quad 4 \quad 1 \quad 3 \quad -9]$$

(3) 열행렬(Column matrix), 열벡터(Column vector)

- $n \times 1$ 행렬

예

$$\begin{bmatrix} 2 \\ -5 \\ 0 \end{bmatrix}, \begin{bmatrix} 2 \\ -5 \\ 0 \\ 7 \end{bmatrix}$$

1) 행렬의 정의와 연산

(4) 대각성분(Diagonal entry)

▪ a_{ii} ($i = 1, 2, \dots$)

예 $\begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix} \rightarrow a_{11} = 1, a_{22} = 5$

(5) 정사각행렬(Square matrix)

▪ $n \times n$ 행렬

예 $\begin{bmatrix} 6 & 7 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 1 \\ 1 & 3 & 3 \\ -8 & 5 & -2 \end{bmatrix}$

(6) 대각행렬(Diagonal matrix)

▪ $n \times n$ 행렬, $a_{ij} = 0$ if $i \neq j$

예 $\begin{bmatrix} 6 & 0 \\ 0 & 1 \end{bmatrix}, \begin{bmatrix} 0 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & -2 \end{bmatrix}$

1) 행렬의 정의와 연산

(7) 단위행렬(Identity matrix)

- I_n ($n \times n$ 행렬, $a_{ii} = 1, (i = 1, 2, \dots)$)인 대각행렬

예 $I_1 = [1], I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

(8) 같다(Equal)

- $A = B$ if A 와 B 는 같은 크기의 행렬 & $a_{ij} = b_{ij}$ (모든 i, j 에 대해서)

예 $A = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}, B = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}$

(9) 덧셈

- $A, B: m \times n$ 행렬일 때, $A + B = [a_{ij} + b_{ij}]$ 로 정의함

예제 $A = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}, B = \begin{bmatrix} 0 & 1 & 2 \\ 4 & 0 & 8 \end{bmatrix}$ 일 때, $A + B$ 는?

풀이 $A + B = \begin{bmatrix} 1 & -2 & 2 \\ 11 & 5 & 17 \end{bmatrix}$

1) 행렬의 정의와 연산

(9) 스칼라곱

- $A: m \times n$ 행렬, $c \in R$ 일 때, $cA = [ca_{ij}]$ 로 정의함

예제 $A = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}$, $c = 2$ 일 때, cA 는?

풀이 $cA = \begin{bmatrix} 2 & -6 & 0 \\ 14 & 10 & 18 \end{bmatrix}$

(10) 정의

- $-A = (-1)A$ 로 정의함
- $A - B = A + (-1)B$ 로 정의함

예제 $A = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}$, $B = \begin{bmatrix} 0 & 1 & 2 \\ 4 & 0 & 8 \end{bmatrix}$ 일 때, $A - B$ 는?

풀이 $A - B = A + (-1)B = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix} + \begin{bmatrix} 0 & -1 & -2 \\ -4 & 0 & -8 \end{bmatrix}$
 $= \begin{bmatrix} 1 & -4 & -2 \\ 3 & 5 & 1 \end{bmatrix}$

1) 행렬의 정의와 연산

(11) 영행렬(Zero Matrix)

- 모든 원소가 0인 행렬

➡ 기호로 O , $O_{m \times n}$ 와 같이 표기함

(12) $A, O : m \times n$ 행렬일 때

- $A + O = O + A = A$
- $A - A = O = (-A) + A$

(13) 행렬의 덧셈과 스칼라배의 대수적 성질

$$A + B = B + A$$

$$(A + B) + C = A + (B + C)$$

$$A + O = A$$

$$A + (-A) = O$$

$$c(A + B) = cA + cB$$

$$(c + d)A = cA + dA$$

$$c(dA) = (cd)A$$

$$1A = A$$

1) 행렬의 정의와 연산

(14) 행렬의 곱셈

- $A: m \times n, B: n \times r$ 일 때, $C = AB$ 는 다음과 같이 정의함

➡ $C = [c_{ij}]$
 $(c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj}, (1 \leq i \leq m, 1 \leq j \leq r))$

예제 $A = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}, B = \begin{bmatrix} 0 & 4 \\ 1 & 0 \\ 2 & 8 \end{bmatrix}$ 일 때, $C = AB$ 는?

풀이 $C = \begin{bmatrix} -3 & 4 \\ 23 & 100 \end{bmatrix}, (c_{11} = 1 \times 0 + (-3) \times 1 + 0 \times 2 = -3)$

(15) 행렬의 거듭제곱

- A 가 정사각행렬일 때, A 의 거듭제곱을 다음과 같이 정의함

➡ $A^1 = A$

➡ $A^k = AA \cdots A \ (k \geq 2) \leftarrow k \text{ 개의 곱}$

예 $A = \begin{bmatrix} 6 & 7 \\ 0 & 1 \end{bmatrix}$ 일 때,
 $A^1 = A = \begin{bmatrix} 6 & 7 \\ 0 & 1 \end{bmatrix}, A^2 = AA = \begin{bmatrix} 6 & 7 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 6 & 7 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 36 & 49 \\ 0 & 1 \end{bmatrix}$

1) 행렬의 정의와 연산

(16) 행렬 곱셈의 성질

$$A(BC) = (AB)C$$

$$A(B+C) = AB+AC$$

$$(A+B)C = AC+BC$$

$$k(AB) = (kA)B = A(kB)$$

$$I_m A = A = A I_n \quad (A : m \times n)$$

1) 행렬의 정의와 연산

(17) 전치행렬(Transpose)

- $A : m \times n$ 행렬이고, $A = [a_{ij}]$ 일 때, A 의 전치행렬을 다음과 같이 정의함

➡ $A^T = [a_{ji}]$ ($n \times m$ 행렬)

예 $A = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}$ 일 때, $A^T = \begin{bmatrix} 1 & 7 \\ -3 & 5 \\ 0 & 9 \end{bmatrix}$ 이다.

예 $u = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}, v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$ 일 때,
 $u \cdot v = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n = u^T v$

1) 행렬의 정의와 연산

(18) 전치행렬의 성질

$$(A^T)^T = A$$

$$(A + B)^T = A^T + B^T$$

$$(kA)^T = kA^T$$

$$(AB)^T = B^T A^T$$

$$(A^r)^T = (A^T)^r, (r = 1, 2, 3, \dots)$$

2) 분할행렬의 연산

(1) 행렬의 분할

- 행렬에 가로선과 세로선을 추가하고 행렬을 분할하여 나타낼 수 있음

예
$$\left[\begin{array}{cc|c} 1 & 2 & 0 \\ 3 & 4 & 0 \\ \hline -1 & -2 & 100 \end{array} \right] = \begin{bmatrix} A & B \\ C & D \end{bmatrix}$$

(2) 행렬-열 표현(Matrix-column representation)

$$A: m \times n, B: n \times r$$

$$AB = A [b_1 : b_2 : \cdots : b_r] = [Ab_1 : Ab_2 : \cdots : Ab_r]$$

예 $A = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}, B = \begin{bmatrix} 0 & 4 \\ 1 & 0 \\ 2 & 8 \end{bmatrix}$ 일 때

$$Ab_1 = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix} = \begin{bmatrix} -3 \\ 23 \end{bmatrix},$$

$$Ab_2 = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix} \begin{bmatrix} 4 \\ 0 \\ 8 \end{bmatrix} = \begin{bmatrix} 4 \\ 100 \end{bmatrix} \text{ 이고}$$

$$AB = \begin{bmatrix} -3 & 4 \\ 23 & 100 \end{bmatrix} \text{ 이다.}$$

2) 분할행렬의 연산

(3) 행-행렬 표현(Row-matrix representation)

$$A: m \times n, B: n \times r$$

$$AB = \begin{bmatrix} A_1 \\ A_2 \\ \vdots \\ A_m \end{bmatrix} B = \begin{bmatrix} A_1 B \\ A_2 B \\ \vdots \\ A_m B \end{bmatrix}$$

예 $A = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}, B = \begin{bmatrix} 0 & 4 \\ 1 & 0 \\ 2 & 8 \end{bmatrix}$ 일 때

$$A_1 B = [1 \quad -3 \quad 0] \begin{bmatrix} 0 & 4 \\ 1 & 0 \\ 2 & 8 \end{bmatrix} = [-3 \quad 4],$$

$$A_2 B = [7 \quad 5 \quad 9] \begin{bmatrix} 0 & 4 \\ 1 & 0 \\ 2 & 8 \end{bmatrix} = [23 \quad 100] \text{ 이고}$$

$$AB = \begin{bmatrix} -3 & 4 \\ 23 & 100 \end{bmatrix} \text{ 이다.}$$

2) 분할행렬의 연산

(4) 행-열 표현(Row-column representation)

$$A: m \times n, B: n \times r$$

$$AB = \begin{bmatrix} A_1 \\ A_2 \\ \vdots \\ A_m \end{bmatrix} [b_1 : b_2 : \cdots : b_r] \leftarrow \text{일반적인 행렬의 곱}$$

예 $A = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}, B = \begin{bmatrix} 0 & 4 \\ 1 & 0 \\ 2 & 8 \end{bmatrix}$ 일 때

$$A_1 b_1 = [1 \quad -3 \quad 0] \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix} = [-3],$$

$$A_1 b_2 = [1 \quad -3 \quad 0] \begin{bmatrix} 4 \\ 0 \\ 8 \end{bmatrix} = [4]$$

$$A_2 b_1 = [7 \quad 5 \quad 9] \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix} = [23],$$

$$A_2 b_2 = [7 \quad 5 \quad 9] \begin{bmatrix} 4 \\ 0 \\ 8 \end{bmatrix} = [100]$$

$$AB = \begin{bmatrix} -3 & 4 \\ 23 & 100 \end{bmatrix} \text{ 이다.}$$

2) 분할행렬의 연산

(5) 열-행 표현(Column-row representation)

$$A: m \times n, B: n \times r$$

$$AB = [a_1 \vdots a_2 \vdots \cdots \vdots a_r] \begin{bmatrix} B_1 \\ B_2 \\ \vdots \\ B_m \end{bmatrix} = a_1 B_1 + a_2 B_2 + \cdots + a_n B_n$$

← 외적전개(Outer product expansion,
 $(a_i B_i \leftarrow$ 외적(Outer product))

예 $A = \begin{bmatrix} 1 & -3 & 0 \\ 7 & 5 & 9 \end{bmatrix}, B = \begin{bmatrix} 0 & 4 \\ 1 & 0 \\ 2 & 8 \end{bmatrix}$ 일 때

$$a_1 B_1 = \begin{bmatrix} 1 \\ 7 \end{bmatrix} \begin{bmatrix} 0 & 4 \end{bmatrix} = \begin{bmatrix} 0 & 4 \\ 0 & 28 \end{bmatrix},$$

$$a_2 B_2 = \begin{bmatrix} -3 \\ 5 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} -3 & 0 \\ 5 & 0 \end{bmatrix},$$

$$a_3 B_3 = \begin{bmatrix} 0 \\ 9 \end{bmatrix} \begin{bmatrix} 2 & 8 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 18 & 72 \end{bmatrix},$$

$$AB = \begin{bmatrix} 0 & 4 \\ 0 & 28 \end{bmatrix} + \begin{bmatrix} -3 & 0 \\ 5 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 18 & 72 \end{bmatrix}$$

$$= \begin{bmatrix} -3 & 4 \\ 23 & 100 \end{bmatrix}$$

2) 분할행렬의 연산

(6) 분할행렬의 덧셈과 스칼라곱

- 블록 단위로 연산 ◀ 직관적으로 당연함

예 $X = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 0 \\ -1 & -2 & 100 \end{bmatrix} = \begin{bmatrix} A & B \\ C & D \end{bmatrix},$

$$Y = \begin{bmatrix} 0 & 2 & -1 \\ 1 & 2 & 1 \\ -1 & 0 & 4 \end{bmatrix} = \begin{bmatrix} E & F \\ G & H \end{bmatrix}$$

$$X + Y = \begin{bmatrix} A + E & B + F \\ C + G & D + H \end{bmatrix}$$

$$2X = \begin{bmatrix} 2A & 2B \\ 2C & 2D \end{bmatrix}$$

2) 분할행렬의 연산

(7) 분할행렬의 곱셈

- 두 분할 행렬의 곱셈은 분할된 블록끼리 곱셈과 덧셈이 정의만 된다면
분할된 블록이 마치 숫자인 것처럼 생각하여 계산할 수 있음

예 $X = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 0 \\ -1 & -2 & 100 \end{bmatrix} = \begin{bmatrix} A & B \\ C & D \end{bmatrix},$

$$Y = \begin{bmatrix} 0 & 2 & -1 \\ 1 & 2 & 1 \\ -1 & 0 & 4 \end{bmatrix} = \begin{bmatrix} E & F \\ G & H \end{bmatrix}$$

$$XY = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 6 & 1 \\ 4 & 14 & 1 \\ -102 & -6 & 399 \end{bmatrix}$$

1) 선형연립방정식과 행렬방정식

(1) 선형연립방정식과 행렬방정식은 같은 것으로 생각할 수 있음

선형연립방정식

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2$$

:

:

$$a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m$$



행렬방정식

$$AX = B$$

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix},$$

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}, B = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

1) 선형연립방정식과 행렬방정식

(1) 선형연립방정식과 행렬방정식은 같은 것으로 생각할 수 있음

예제1 다음 선형연립방정식을 행렬방정식으로 나타내면?

$$x + y - z = 1$$

$$2x + y + z = 0$$

$$x - 3y = 4$$

풀이 $AX = B$

$$A = \begin{bmatrix} 1 & 1 & -1 \\ 2 & 1 & 1 \\ 1 & -3 & 0 \end{bmatrix}, X = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, B = \begin{bmatrix} 1 \\ 0 \\ 4 \end{bmatrix}$$

예제2 다음 행렬방정식을 행렬방정식으로 나타내면?

$$AX = B$$

$$A = \begin{bmatrix} 1 & 1 & -1 \\ 2 & 1 & 1 \\ 1 & -3 & 0 \end{bmatrix}, X = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, B = \begin{bmatrix} 1 \\ 0 \\ 4 \end{bmatrix}$$

풀이 $x + y - z = 1$

$$2x + y + z = 0$$

$$x - 3y = 4$$

2) 역행렬

(1) 정의

- $A(n \times n \text{ 행렬}), I(n \times n \text{ 단위 행렬}), B(n \times n \text{ 행렬})$ 에 대하여,
 $AB = BA = I$ 일 때 B 를 A 의 역행렬이라고 함

➡ 이 때, A 를 가역(Invertible)행렬이라고 부름

예 $A = \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix}, B = \begin{bmatrix} -1 & 2 \\ 1 & -1 \end{bmatrix}, I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ 일 때,
 $AB = BA = I$ 임

➡ B 는 A 의 역행렬이고, A 는 가역행렬임

(2) 역행렬의 유일성

- A 가 가역행렬이면 A 의 역행렬은 유일함
- A 의 역행렬이 존재할 때, 그 역행렬을 A^{-1} 로 표기함

2) 역행렬

(3) 2X2 행렬의 역행렬

▪ $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 에 대하여,

A 가역 $\iff ad - bc \neq 0$

➡ 이 때, $A^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$

➡ $\det(A) = ad - bc$ 로 정의함

예제 $A = \begin{bmatrix} 1 & 2 \\ 3 & 5 \end{bmatrix}$ 의 역행렬은?

풀이 $\det(A) = 1 \times 5 - 2 \times 3 = -1$

$$A^{-1} = \frac{1}{-1} \begin{bmatrix} 5 & -2 \\ -3 & 1 \end{bmatrix} = \begin{bmatrix} -5 & 2 \\ 3 & -1 \end{bmatrix}$$

2) 역행렬

(4) 가역행렬의 성질

$$A: \text{가역} \Rightarrow A^{-1}: \text{가역} \ \& \ (A^{-1})^{-1} = A$$

$$A: \text{가역}, c \neq 0, c \in R \Rightarrow cA: \text{가역} \ \& \ (cA)^{-1} = \frac{1}{c} A^{-1}$$

$$A, B: n \times n \text{ 가역} \Rightarrow \text{가역}, (AB)^{-1} = B^{-1}A^{-1}$$

$$A: \text{가역} \Rightarrow A^T: \text{가역}, (A^T)^{-1} = (A^{-1})^T$$

$$A: \text{가역} \Rightarrow A^n: \text{가역}, (A^n)^{-1} = (A^{-1})^n, (n = 1, 2, 3, \dots)$$

2) 역행렬

(5) 역행렬을 이용한 선형연립방정식의 풀이

- 선형연립방정식 $\Rightarrow$ 행렬방정식
- $AX = B, A^{-1}$ 존재할 때 $\Rightarrow X = A^{-1}B$

예제 다음 선형연립방정식을 행렬방정식과 역행렬을 이용하여 나타내면?

$$x + 2y = 1$$

$$3x + 5y = 4$$

풀이 $A = \begin{bmatrix} 1 & 2 \\ 3 & 5 \end{bmatrix}, X = \begin{bmatrix} x \\ y \end{bmatrix}, B = \begin{bmatrix} 1 \\ 4 \end{bmatrix}$ 일 때,

문제의 선형연립방정식은 $AX = B$ 임

$$A^{-1} = \begin{bmatrix} -5 & 2 \\ 3 & -1 \end{bmatrix} \text{이므로}$$

$$X = A^{-1}B = \begin{bmatrix} -5 & 2 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

따라서, $x = 3, y = -1$

3) 영공간

(1) 역행렬을 이용한 선형연립방정식의 풀이

- $A(m \times n \text{ 행렬})$ 에 대하여 $Null(A) = \{X \mid AX = 0\}$ 은 R^n 의 부분공간임

➡ 이 때, $Null(A)$ 를 A 의 영공간(Null Space)이라고 함

예 $A = \begin{bmatrix} 1 & 2 \\ -2 & -4 \end{bmatrix}$ 일 때,

$$Null(A) = \left\{ \begin{bmatrix} x \\ y \end{bmatrix} \mid \begin{bmatrix} 1 & 2 \\ -2 & -4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \right\} = \{(-2t, t) \mid t \in R\}$$